



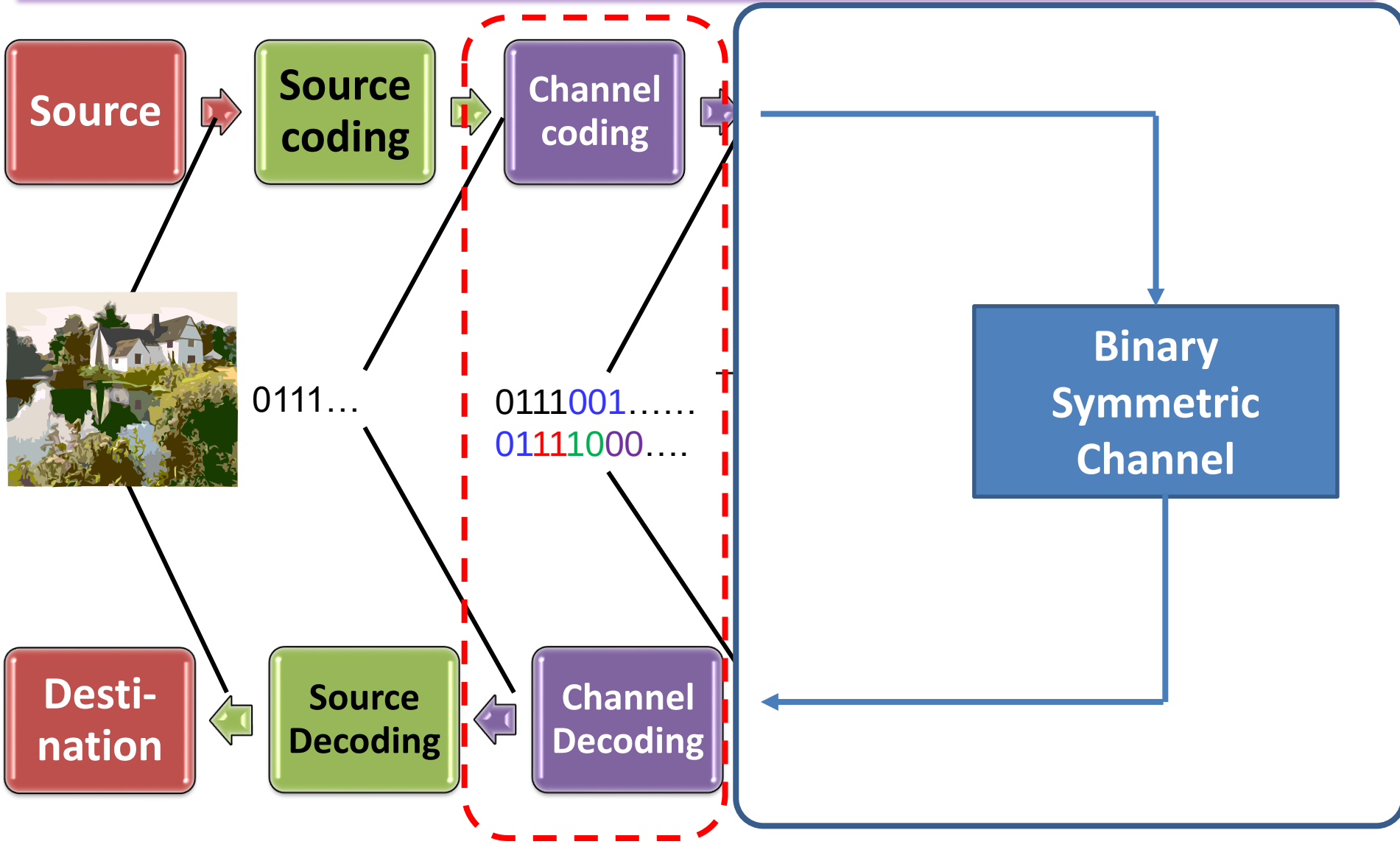
Communications and Networks

Lecture 8: Error Correction Coding

Sheng Zhou



Design Perspective of Channel Coding

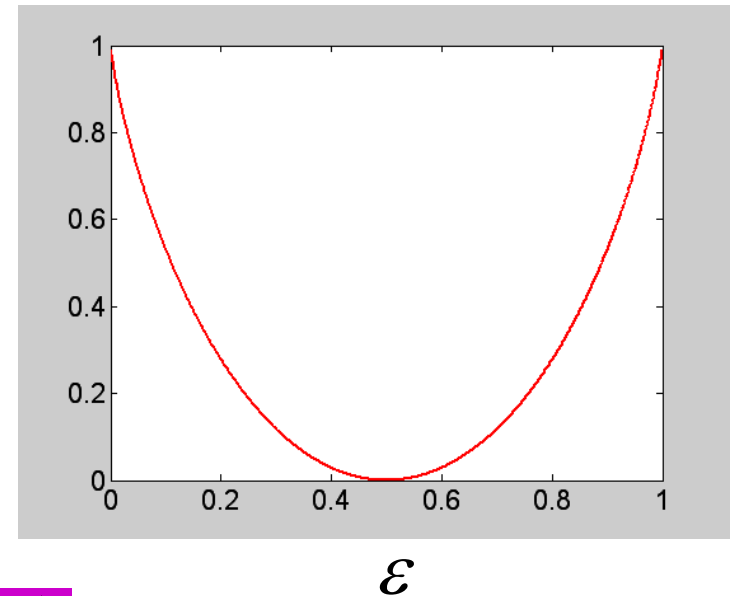
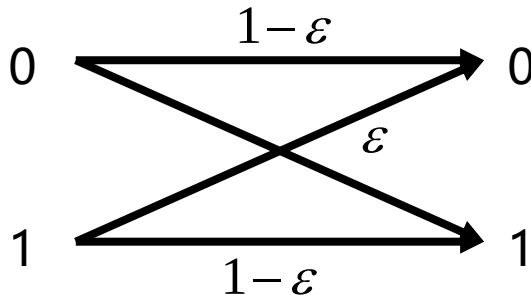




Recall the Binary Symmetric Channel

■ Binary Symmetric Channel (BSC)

- ▶ A typical model for communication channel
- ▶ Applied in analyzing the channel coding
- ▶ Basic model:



$$C = 1 + \varepsilon \log \varepsilon + (1 - \varepsilon) \log(1 - \varepsilon)$$



Motivations

- Noise → limited capacity of the channel
 - ▶ Due to errors in transmitting bits ($0 \rightarrow 1; 1 \rightarrow 0$)
- A fundamental conflict in digital transmission
 - ▶ Transmission errors caused by the noise
 - ▶ Reliability in data transmission
- However, noise is inevitable in the channel

Can we transmit message reliably, reaching the channel capacity?

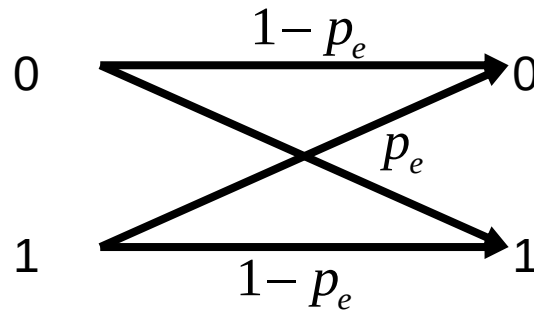
Or at least reduce the bit error rate?



Channel Coding: An Example

■ Repetition Code

► Binary Symmetric Channel



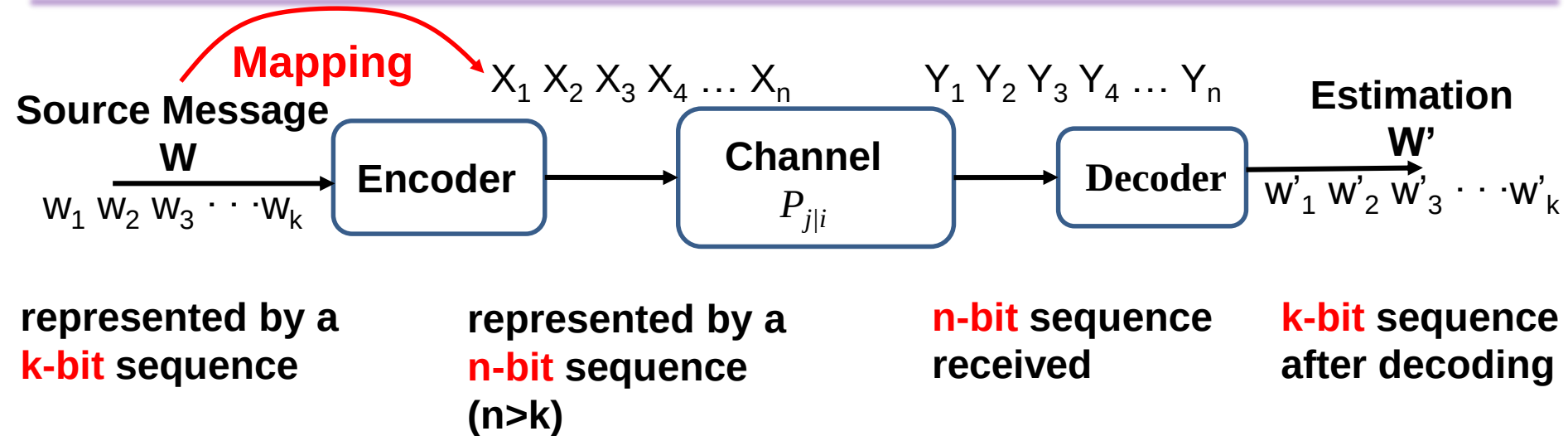
Cost: redundancy
reduces efficiency

- 1 bit transmitted once, error probability: p_e
- 1 bit transmitted three times. '1' for more than two '1' received, '0' for more than two '0' received
- Then the error probability: $[3p_e^2(1 - p_e) + p_e^3] \approx 3p_e^2$

Reduce error probability significantly!



Channel Coding: WHAT?



■ Channel Coding :

- ▶ By properly adding redundancies, channel coding corrects possible errors in channel transmission
- ▶ Also known as, **Error Correction Coding**



Ideal Channel Coding

- Shannon's channel coding theorem
- For a noise channel with capacity C
 - ▶ For any transmission rate $R < C$
 - ▶ There exists a coding method so that

Error Probability $P \leq e^{-nE(R)}$

n : code length, $E(R)$ is the error index



For reliable transmission, the code length goes to infinity!



Practical Channel Coding

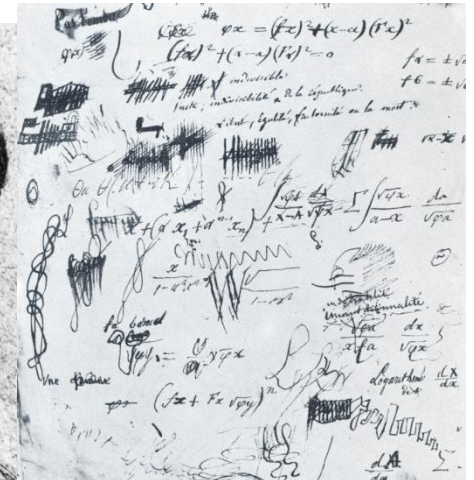
- Why not use ideal channel coding?
- Two reasons:
 - ▶ The code length goes to infinity
 - ▶ No algebraic structures, with too high complexity
- How to design a practical code?
 - ▶ **Finite** code length — Cost: non-zero error probability
 - ▶ With algebraic structures — Benefit: easy to decode
- Metrics
 - ▶ Bit Error Rate (BER) — **Reliability**
 - ▶ Code Rate — **Efficiency** ($R=k/n$)



Channel Coding: History

■ History of channel coding

- ▶ In 1950, R. Hamming from Bell Lab, a colleague of Shannon, proposed the first structured error correcting code.
- ▶ Based on the group theory, invented by Galois (1811~1832). Algebra is widely applied in error correction code.





Categories of Channel Coding

- Three categories :
 - ▶ Detection with Feedback
 - ▶ Error Detection (ARQ)
 - ▶ Forward Error Correction (FEC)
- At present, FEC is well researched with mature theory
 - ▶ Linear Code vs Non-linear Code
 - ▶ Block Code vs Convolutional Code
 - ▶ Systematic Code vs Non-systematic Code

We will clarify them later!



Prerequisite Mathematics

■ Galois Field

Galois Field (2)

► We focus on binary code, using GF(2)

► Rules:

Addition

$$0 + 0 = 0$$

$$0 + 1 = 1 + 0 = 1$$

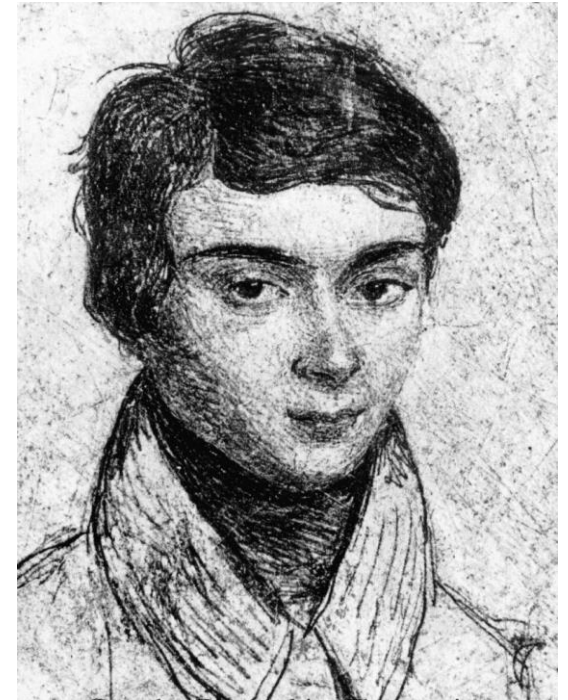
$$1 + 1 = 0$$

Multiplication

$$0 \times 0 = 0$$

$$0 \times 1 = 1 \times 0 = 0$$

$$1 \times 1 = 1$$





Block Code

■ Block Code

- ▶ Definition: The redundancy bits added only depend on the information bits
 - Specifically, the bit sequence is split into segments of length k . Each segment is mapped to a codeword of length n , according to the codebook
 - 0 0 | 1 0 | 0 1 | 1 1 $k=2$
 - 000 | 101 | 011 | 110 $n=3$
- ▶ Advantage: easy to implement and analyze
 - Easy to build a systematic mathematical theory
 - Easy to design low-complexity encoders/decoders



Block Code: Concepts

■ Parity Bits

- ▶ The code length after coding is n , among which k bits are information bits, the other $r=n-k$ bits are parity bits

■ Systematic Code

- ▶ The information bits are the first k bits, and the parity bits are the last $r=n-k$ bits
- ▶ Practical channel codes are usually systematic

■ Valid Codewords

- ▶ There are 2^k distinct valid codewords of length n mapped from the k information bits (WHY?)



A Simple Example

■ Error Detection Code

▶ Note that we **only detect** errors here

▶ Parity Bit (with code length n)

$$a_1 a_2 \cdots a_{n-1} a_n$$

There must be errors if the sum is 1

- Take the sum of the $n-1$ information bits to obtain the parity bit:

$$a_n = a_1 + a_2 + \cdots + a_{n-1}$$

- Detection: Correctly received codeword satisfies:

$$\begin{aligned} & a_1 + a_2 + \cdots + a_{n-1} + a_n \\ &= a_1 + a_2 + \cdots + a_{n-1} + a_1 + a_2 + \cdots + a_{n-1} \\ &= 0 \end{aligned}$$



Parity Bit

■ Error Detection Capability

▶ Does zero sum guarantees correct receiving?

● 0 0 | 1 0 | 0 1 | 1 1 —>

● 000 | 101 | 011 | 110

● —> Transmission —>

● 000 | 111 | 101 | 000

Detection Failure

Not always! Cannot detect an even number of errors.

▶ Only an odd number of errors can be detected

● In fact, can only detect *no more than one* error



A More Complex Example

■ Counter Code (Error Detection)

▶ Put the number of ‘1’s in the information bits at the end of the code

▶ Examples

● 0 0 0 | 1 0 0 | 0 1 1 | 1 1 1 ———>

● 00000 | 10001 | 01110 | 11111

▶ Error detection capability

● Better than the parity bit

● Cannot detect when {num of ‘1’ → ‘0’ = num of ‘0’ → ‘1’ }

What else situations?



Intuition of Channel Coding

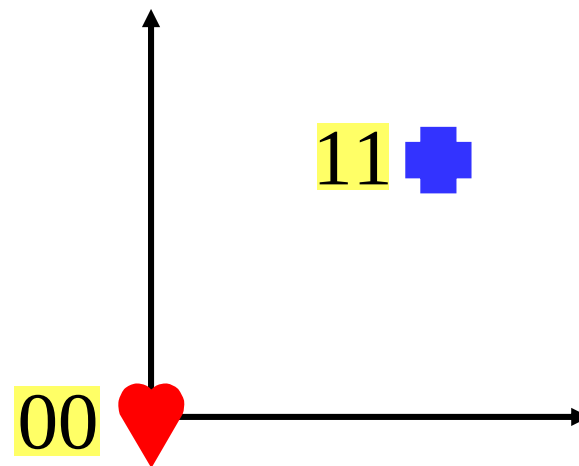
■ Geometric representation of codewords

- ▶ As we have seen, the valid codewords in binary block code are binary sequences of length n
- ▶ Intuitively, a codeword corresponds to a point in the n -dimension space

■ Definition of the **Distance**

- ▶ Example with 00 & 11:

- Euclidean distance $\sqrt{2}$
- **Hamming distance** 2



Hamming distance: the number of different bits between codewords



Hamming Distance

■ Hamming distance is widely used in channel coding

▶ Relates to detection & probability theory

▶ Example: consider the codeword 000

● Receive 001, Hamming distance: 1

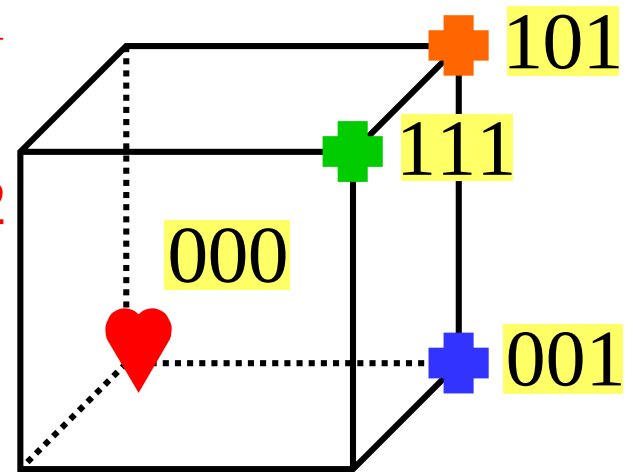
Probability: $(1 - p_e)^2 p_e$

● Receive 101, Hamming distance: 2

Probability: $(1 - p_e) p_e^2$

● Receive 111, Hamming distance: 3

Probability: p_e^3

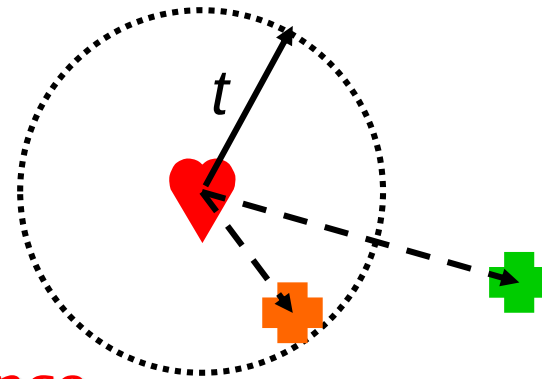


Distance: The least number of edges to pass on the cube



Hamming Distance & Detection

- Obviously, the larger Hamming distance, the smaller probability that one codeword is mistaken for another ($P_e < 0.5$)



- Detection Rule:

- ▶ **Minimum Hamming Distance**

- ▶ If the distance to a codeword is:

- less or equal than the threshold t , then that codeword is decided

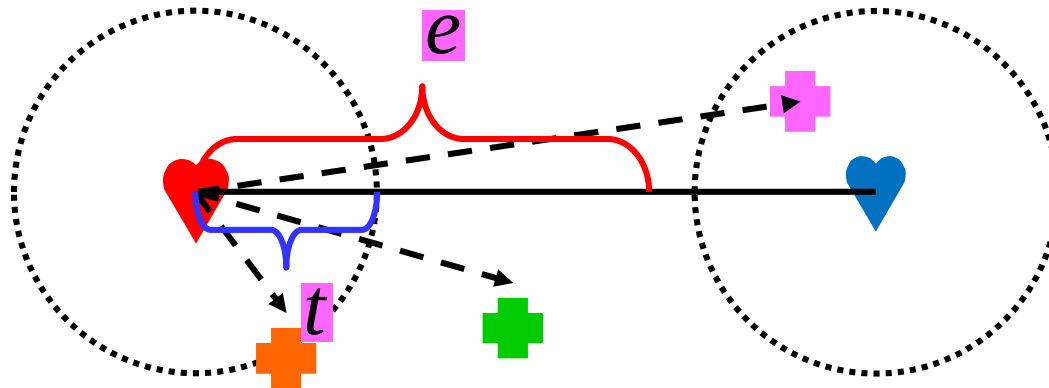
Error correction works as long as the number of errors is no more than t



Error Correction and Detection

■ Graphical Interpretation:

- ▶ Consider two codewords that have the minimum Hamming distance $d_{\min}$ in the codebook:



What if the distance to all codewords is larger than t ?

We only know there must be errors, but cannot correct them.

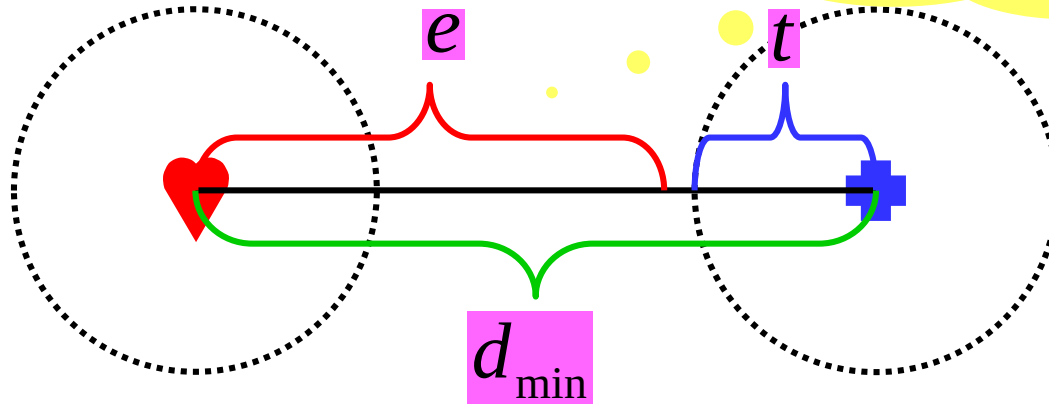
The maximum bits for error detection and correction is e & t .

What is the relationship between e and t ?



Error Correction and Detection

■ Graphical Interpretation:

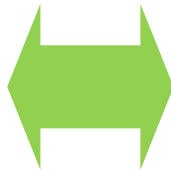


Obviously, the larger the minimum distance, the better performance

$$d_{\min} \geq t + e + 1$$

$$t \leq e$$

$$t = 0$$



$$d_{\min} \geq e + 1$$

$$t = e$$



$$d_{\min} \geq 2t + 1$$

Why there is a 1?



Design of Channel Coding

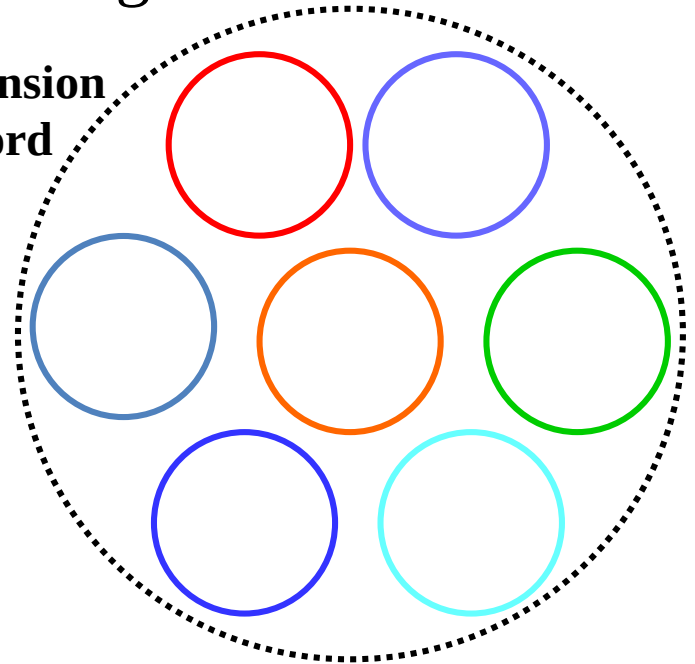
■ Two Principles:

- ▶ **Reliability**: Maximize the minimum distance
- ▶ **Efficiency**: Maximize the number of valid codewords in the codebook, given the code length n

■ Efficiency

- ▶ Code rate: k/n
- ▶ k : num of information bits
- ▶ n : num of codeword length
- ▶ **Valid codeword** 2^k

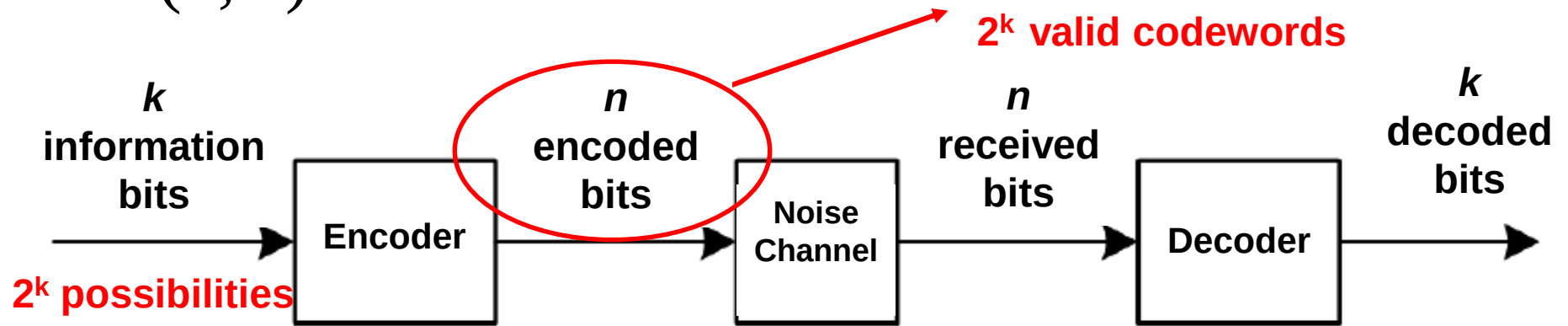
n-dimension
codeword
space





Summary of Block Code

■ (n, k) block code



▶ Efficiency: k/n

■ The minimum Hamming distance $d_{\min}$

▶ Error detection capability: e (in bits),

▶ Error correction capability t (in bits)

$$d_{\min} \geq t + e + 1 \quad t \leq e$$



Linear Block Code

- Maximize the minimum codeword distance in a codebook

- ▶ Symmetry: construct a group for codewords

- Design of linear block code

- ▶ Encoding: Multiply the generation matrix $\mathbf{G}$

$$\mathbf{a} = \mathbf{x}\mathbf{G} = \mathbf{x}[\mathbf{I}:\mathbf{Q}]$$

n -dimension
codeword

k -dimension
information vector

$\mathbf{G}$ is usually full-rank. If $\mathbf{G}$ is in the form $[\mathbf{I}:\mathbf{Q}]$, then it is a systematic code

Note: calculations are based on GF(2)



Encoding in Linear Block Code

- The valid codewords in linear block code forms a group

$$\mathbf{a}_1 + \mathbf{a}_2 = \mathbf{x}_1 \mathbf{G} + \mathbf{x}_2 \mathbf{G} = (\mathbf{x}_1 + \mathbf{x}_2) \mathbf{G}$$

- Linear block code is usually a systematic code

$$\mathbf{a} = \mathbf{x}[\mathbf{I}:\mathbf{Q}]$$

Systematic Code: Because of the identity matrix, the information bits are copied as the first part of the codeword

- The width of $\mathbf{Q}$ is the number of parity bits r , with efficiency:

$$\frac{k}{n} = \frac{k}{k+r}$$



Decoding in Linear Block Code

■ Similarly, we decode according to the encoding

► Decoding

$$\mathbf{a} = \mathbf{x}[\mathbf{I}:\mathbf{Q}]$$

Note the dimension
of the matrix

► Note the important identity below:

Calculations on GF(2)

$$\begin{aligned} \mathbf{a} \begin{bmatrix} \mathbf{Q} \\ \mathbf{I} \end{bmatrix} &= \mathbf{x}[\mathbf{I}_k : \mathbf{Q}] \begin{bmatrix} \mathbf{Q} \\ \mathbf{I}_r \end{bmatrix} \\ &= \mathbf{x}(\mathbf{Q} + \mathbf{Q}) = \mathbf{x}\mathbf{0} = \mathbf{0} \end{aligned}$$

$$r = n - k$$



Error **Detection** in Linear Block Coding

- If the above identity is not satisfied, i.e.

$$\mathbf{b} \begin{bmatrix} \mathbf{Q} \\ \mathbf{I}_r \end{bmatrix} \neq \mathbf{0}$$

- Then $\mathbf{b}$ is not a valid codeword! There must be errors.
- Thus the decision rule of error detection is

$$\mathbf{bH}^T \neq \mathbf{0}$$

- $\mathbf{H} = [\mathbf{Q}^T : \mathbf{I}_r]$ is the **typical parity check** matrix



Error Correction in Linear Block Coding

- Linear code can not only detect errors, but also correct errors
- The received sequence is represented by $\mathbf{b} = \mathbf{a} + \mathbf{e}$

▶ where
$$\mathbf{e}(i) = \begin{cases} 0 & \mathbf{a}(i) = \mathbf{b}(i) \\ 1 & \mathbf{a}(i) \neq \mathbf{b}(i) \end{cases}$$

- Calculate the syndrome

$$\mathbf{bH}^T = (\mathbf{a} + \mathbf{e})\mathbf{H}^T = \mathbf{aH}^T + \mathbf{eH}^T = \mathbf{eH}^T$$

a r -dimension vector



Error Correction in Linear Block Coding

- Basic idea: use the **syndrome** to indicate the error positions, to achieve error correction
- Can identify $2^r - 1$ error patterns (WHY?)
 - ▶ Each syndrome corresponds to one error pattern
 - ▶ Example: Suppose there is **only one bit error**
 - Since the codeword length is n , n different error patterns need to be identified.
 - The syndrome length is r , and thus the syndrome can represent at most 2^r patterns
 - Therefore, for a linear block code with codeword length n , the syndrome length should be at least $\lceil \log_2(n+1) \rceil$



Error Correction in Linear Block Coding

■ Consider the ideal situation

▶ Exactly, $2^r = n + 1$

▶ The syndrome with length r correspond to the n error patterns, indicating where the error bit is

$$\mathbf{s} = \mathbf{e}\mathbf{H}^T$$

▶ If the v -th bit has an error, the syndrome is

$$\mathbf{s} = \mathbf{H}^T(v, :)$$

Decide the error position according to $\mathbf{s}$

▶ i.e. the v -th row of the transpose of parity check matrix



Error Correction in Linear Block Coding

■ Design of the Parity Check Matrix

- ▶ Obviously, not every matrix can be a parity check
- ▶ Should satisfies: rows are different after transpose
- ▶ **Columns of H are different**
- ▶ Recall the transpose of the parity check matrix

$$\mathbf{H}^T = \begin{bmatrix} \mathbf{Q} \\ \mathbf{I}_r \end{bmatrix}$$

Rows of Q reflects which information bit is wrong

There are at least two '1's in each row of Q (why?)

If the syndrome has only one '1', then a parity bit has an error (why?)

What if the syndrome is all zero?



Summary of Linear Block Code

■ Generation and Decoding of (n, k) Linear Block Code

- ▶ Generate the codeword: $\mathbf{a} = \mathbf{xG}$
- ▶ Codewords are in a k -dim subspace of n -dim vector space
- ▶ Rows of generator matrix $\mathbf{G}$ is a basis of valid codeword space
- ▶ Parity check matrix $\mathbf{H}$ can be constructed by a basis of the null space of the valid codeword space

If the syndrome $\mathbf{bH}^T$ is all zero, then $\mathbf{b}$ is a valid codeword, otherwise this vector with length r indicates the error position

Identifies $2^r - 1$ unique error patterns

■ For systematic code

- ▶ Generator: $\mathbf{G} = [\mathbf{I} : \mathbf{Q}]$
- ▶ Typical Parity Check: $\mathbf{H} = [\mathbf{Q}^T : \mathbf{I}]$

Rows of $\mathbf{Q}$ reflects which information bit is wrong



Non-Systematic to Systematic

$$\mathbf{G} = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix}$$

Add Row 2
to Row 1



$$\begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix}$$

Add Row 3
to Row 1



$$\begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix}$$

Add Row 3
to Row 2



$$\mathbf{G}_{\text{sys}} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix}$$

$$\mathbf{G} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} \quad \mathbf{G}_{\text{sys}} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} [\mathbf{I} : \mathbf{Q}]$$

$$\mathbf{H} = [\mathbf{Q}^T : \mathbf{I}]$$

Still the parity check
matrix for $\mathbf{G}$?



Example Design of Linear Block Code

- Hamming code: Linear block code satisfying

$$2^r = n + 1$$

- Below we design a Hamming code:

- ▶ Let $r=3$, the codeword length $n = 2^3 - 1 = 7$

- ▶ Length of information bits: $k=n-r=4$

- ▶ Determine the dimensions of the transposed parity check matrix

$$\mathbf{H}^T = \begin{bmatrix} & \mathbf{Q} & \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



Example Design of Linear Block Code

■ Determine Q

▶ Select rows different from **000**, 100, 010, 001

▶ If we choose:

Note that the mapping
here is arbitrary

- 111 to represent error on 1st bit
- 110 to represent error on 2nd bit
- 101 to represent error on 3rd bit
- 011 to represent error on 4th bit

▶ Then

$$Q = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$$



$$H^T = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



Example Design of Linear Block Code

■ Determine the generator by $\mathbf{Q}$

$$[\mathbf{I}:\mathbf{Q}] = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}$$

Now everyone can design a Hamming code!



Properties of Hamming Code

■ Codeword length $n = 2^r - 1$

■ Parity bits r

■ Information bits $k = n - r = 2^r - 1 - r$

■ **Error correction capability $t=1$ (by its design)**

What is the error detection capability?

■ Code rate: $\frac{2^r - 1 - r}{2^r - 1}$

Will the code rate approach 1 by increasing r ?



Error Detection of Hamming Code

■ Minimum distance: $d_{\min} = 3$

■ Error detection capability $e=1$

► From this inequality: $1 = t \leq e \leq d_{\min} - t - 1 = 1$

The capabilities, measure in bits, are the same for error correction and detection

Without re-transmission, error detection is of little use

Therefore, FEC should maximize the error correction capability given the minimum distance



Example Design of Linear Block Code

■ Hamming code: Linear block code satisfying

$$2^r = n + 1$$

► Design a (7,4) Hamming code

⊕ **Determine Q**

□ Select rows different from 000, 100, 010, 001

□ If we choose:

□ 101 for 1st bit error

□ 111 for 2nd bit error

□ 110 for 3rd bit error

□ 011 for 4th bit error

$$\mathbf{G} = [\mathbf{I}_k \quad \mathbf{Q}] = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}$$



Example Design of Linear Block Code

$$\mathbf{G} = [\mathbf{I}_k \quad \mathbf{Q}] = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}$$

Given the input sequence $\mathbf{m}=[0101]$,
we obtain the codeword by the generator

$$\mathbf{A} = \mathbf{XG} = [0101] \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix} = [0101100]$$

$$\mathbf{H}^T = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



Example Design of Linear Block Code

- What is the decoded output if receiving $c=(1001001)$?
- What is the decoded output if receiving $c=(0101000)$?



Example Design of Linear Block Code

- The decoded output if receiving $c=(1001001)$:

$$s = cH^T = [1001001] \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = [111] = EH^T$$

$(1001001) \rightarrow (1101001) \rightarrow (1101)$

Systematic
Code



Example Design of Linear Block Code

- The decoded output if receiving $c=(0101000)$:

$$S = CH^T = [0101000] \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = [100] = EH^T$$

**Error on
a parity bit**

$(0101000) \rightarrow (0101100) \rightarrow (0101)$



Improvement to Linear Block Code

■ Why needs improving?

- ▶ Linear codes are suitable for correcting **sparse** errors
- ▶ Hamming code is helpless against two or more errors
 - If the errors always occurs in pairs, the Hamming code does not work at all
- ▶ In communication systems, there are usually **burst** errors
- ▶ For example: errors caused by fading in wireless channel
- ▶ How to combat errors smartly?

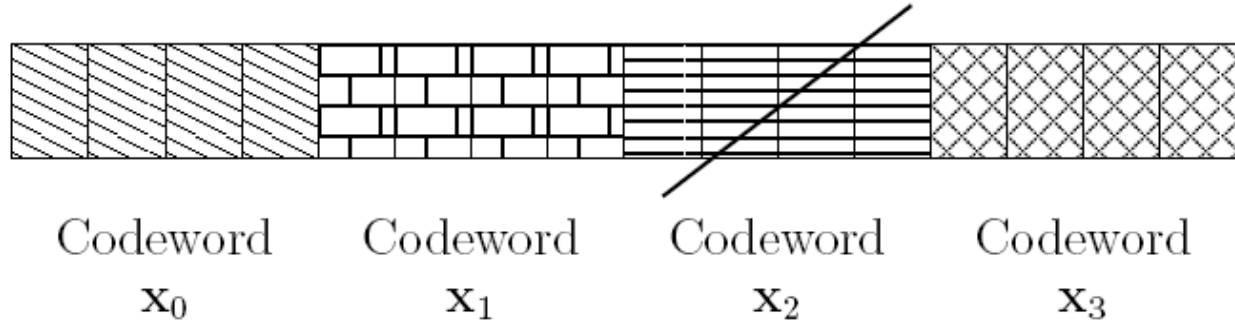
Don't put all your eggs into one basket!



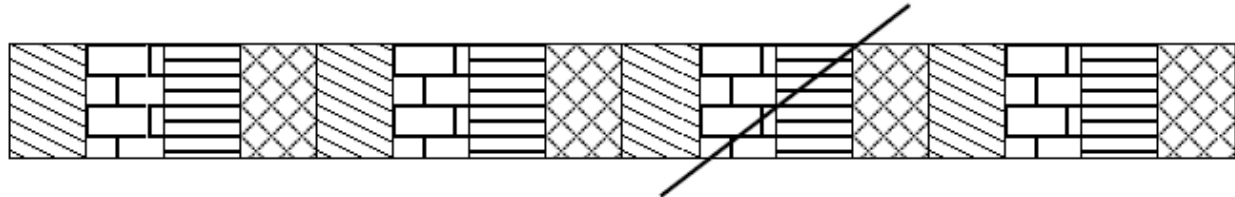
Method to Combat Burst Errors

■ Convert **burst errors** to **sparse errors**

No Interleaving



Interleaving





Interleaver

Basic idea:

- ▶ To combat the burst channel errors, the interleaver alters the sequence of transmitting bits
- ▶ Maximize the time ‘distance’ of originally adjacent bits
- ▶ Example: Consider a (n, k) block code, the interleaved result:

Information bits,
grouped in k bits

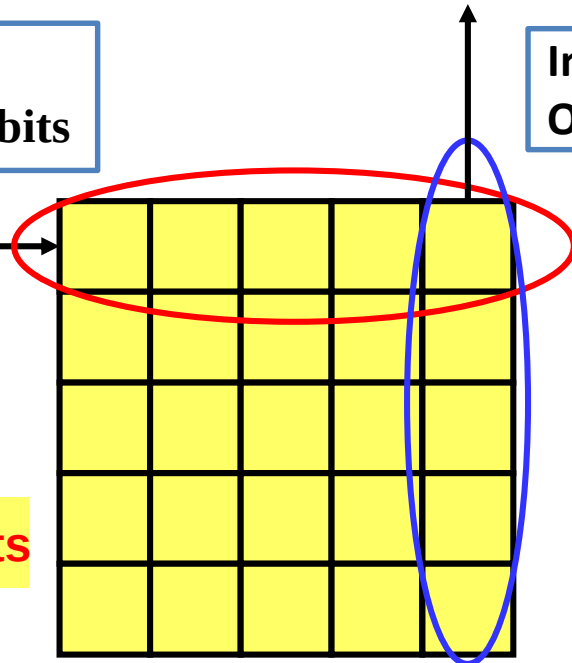
Codewords,
grouped in n bits

Interleaving
Outputs

(n, k) Block Encoder

The interleaver writes horizontally,
reads vertically

Generate gaps between originally adjacent bits





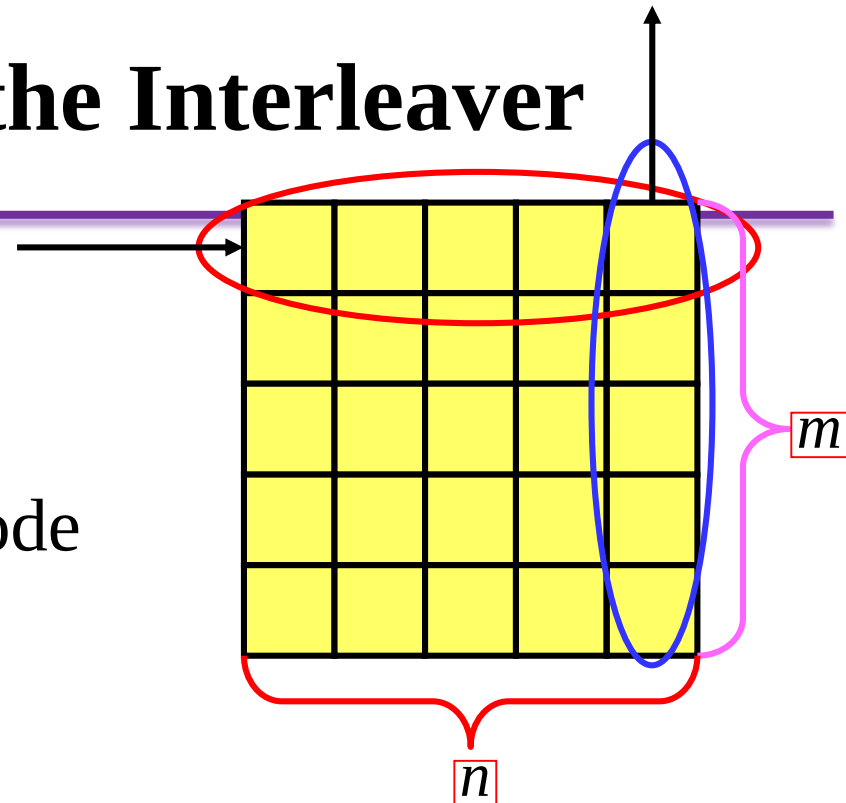
Performance of the Interleaver

■ Width

- ▶ Codeword length n
- ▶ Depend on the specific code

■ Length

- ▶ Decide the gaps between originally adjacent bits
- ▶ m : Interleaving length
- ▶ If the block code can correct b burst errors, then it can correct mb burst errors after interleaving



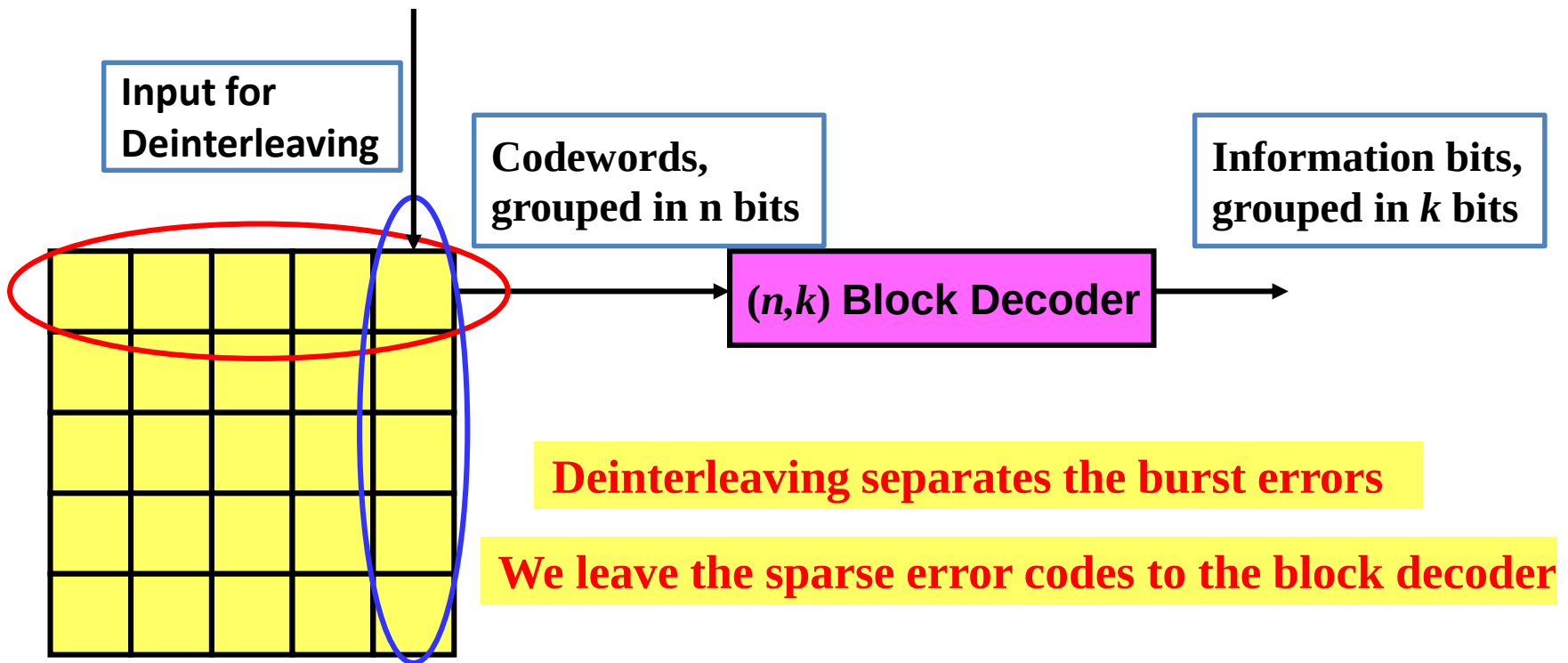
The larger interleaving length is, the stronger capability in correcting burst errors

However, the interleaving length causes decoding latency



De-interleaving

- De-interleaving is required at the receiver
 - ▶ Conversely, **writes vertically, reads horizontally**





Applications of the Interleaver

■ Channel Coding

- ▶ Concatenated code
- ▶ Turbo code

■ Wireless communication systems

- ▶ GSM
- ▶ IS-95
- ▶ WCDMA
- ▶ DVB-S
- ▶ DVB-C



Summary

■ Linear Block Code (Haykin: 10.3)

- ▶ k : Number of information bits; n : codeword Length
- ▶ $r=n-k$: Number of parity bits
- ▶ The linear block code that can correct $t=1$ bit error: $n \leq 2^r - 1$

- Hamming code $n=2^r-1$

- ▶ Generation Matrix: $\mathbf{G} = [\mathbf{I} : \mathbf{Q}]$

Encoding: $\mathbf{a} = \mathbf{xG}$

- ▶ Parity-Check Matrix: $\mathbf{H} = [\mathbf{Q}^T : \mathbf{I}]$

Syndrome: $\mathbf{bH}^T$

■ Interleaving